

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	Yuvasri.s	Reg.No.:	192521032
Department:	B.TECH IT	Date:	30\07\2026

ANNEXURE A

- Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

ANSWER :

A system was designed and simulated using four registers (**R1, R2, R3, and R4**) and an **Arithmetic Logic Unit (ALU)** to perform register transfer, arithmetic, and logical operations. Initially, the registers were loaded with sample values: **R1 = 10 (1010₂)** and **R2 = 6 (0110₂)**. First, the data from **R1** was transferred to **R3** (**R3 ← R1**), demonstrating a register transfer operation. Next, the ALU performed arithmetic operations: **Addition (R4 ← R1 + R2 = 16)** and **Subtraction (R4 ← R1 - R2 = 4)**. It then executed logical operations: **AND (R4 ← R1 AND R2 = 2)** and **OR (R4 ← R1 OR R2 = 14)**. During execution, data was transferred from the source registers to the ALU, where the selected operation was carried out, and the result was stored back in the destination register. This sequence illustrates how registers temporarily hold data while the ALU performs processing. The ALU serves as the core computational unit of the processor, executing arithmetic and logical functions based on control signals from the control unit. The simulation demonstrates efficient data movement between registers, correct execution of ALU operations, and the coordination between the control unit, registers, and ALU in a processor.

The simulation clearly shows how register transfer operations and ALU functions work together inside a processor. Registers act as high-speed storage locations that temporarily hold data and instructions during execution. When an instruction is issued, the control unit selects the source registers, transfers the data to the ALU, and specifies the required operation such as addition, subtraction, AND, or OR. After processing, the ALU sends the result back to the destination register for further use. Using multiple

registers reduces memory access time and improves execution speed. The sequence of register transfers also demonstrates the fetch–decode–execute cycle followed by the CPU. Therefore, the ALU and register organization are essential for efficient data processing in computer systems.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

ANSWER :

A computer system was modeled using both **single-bus** and **multi-bus** organizations to perform data transfer between **registers, memory, and input/output (I/O) devices**. In the **single-bus organization**, all registers, memory, and I/O devices share one common communication bus. Since only one data transfer can occur at a time, operations such as reading data from memory, transferring it to a register, and sending it to the ALU are executed sequentially. This results in increased execution time due to bus contention and waiting periods. For example, transferring data from **Memory** → **R1**, **R1** → **ALU**, and **ALU** → **R2** requires multiple clock cycles because each transfer must wait until the shared bus becomes available.

In the **multi-bus organization**, separate buses are provided for different data paths, allowing multiple transfers to occur simultaneously. For instance, one bus can transfer data from **memory to a register** while another transfers data between **registers and the ALU**, and a third handles **I/O communication**. This parallelism significantly reduces the total execution time and improves overall system throughput. Since several operations can be performed concurrently, the processor spends less time waiting for bus availability, resulting in faster instruction execution and better hardware utilization.

When comparing both approaches, the **single-bus system** is simpler, less expensive, and easier to design but suffers from performance bottlenecks because all components compete for the same communication path. In contrast, the **multi-bus system** is more complex and costly due to additional hardware, but it provides much higher efficiency by supporting parallel data transfers and minimizing delays. Therefore, modern processors commonly use **multi-bus architectures** because they improve data transfer speed, reduce bottlenecks, enhance CPU performance, and efficiently support high-speed computing applications.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

A **5-stage instruction pipeline** consisting of **Instruction Fetch (IF)**, **Instruction Decode (ID)**, **Execute (EX)**, **Memory Access (MEM)**, and **Write-back (WB)** was simulated using a set of instructions such as **LOAD, ADD, SUB, STORE, and AND**. In the pipelined system, each instruction moves to the next stage every clock cycle while a new instruction enters the pipeline simultaneously. For example, while the first instruction is in the Execute stage, the second instruction is in Decode, the third is in Fetch, and the remaining instructions continue in parallel. This overlapping of instruction execution reduces idle CPU time and increases processor efficiency. After the initial pipeline is

filled, one instruction is completed in every clock cycle, significantly improving overall performance.

For comparison, the same instruction set was executed on a **non-pipelined processor**, where each instruction completed all five stages before the next instruction began. Assuming **5 instructions**, the non-pipelined system required **25 clock cycles (5×5)**, whereas the pipelined system completed execution in **9 clock cycles ($5 + 5 - 1$)**. The **speedup** is calculated as $25 \div 9 \approx 2.78$, indicating that the pipelined processor executes the instruction set nearly **2.8 times faster** than the non-pipelined system. The **throughput** also increased from **0.2 instructions per cycle ($5/25$)** in the non-pipelined system to approximately **0.56 instructions per cycle ($5/9$)** in the pipelined system.

The simulation demonstrates that pipelining greatly enhances processor performance by allowing multiple instructions to execute simultaneously at different stages. Although hazards such as data, control, and structural dependencies may introduce stalls, techniques like forwarding, hazard detection, and branch prediction help minimize these delays. Overall, pipelining increases instruction throughput, improves CPU utilization, reduces execution time, and forms the foundation of modern high-performance processor architectures.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

ANSWER :

Pipeline Hazards and Superscalar Execution

A pipeline execution scenario was developed using a **5-stage pipeline (Fetch, Decode, Execute, Memory, and Write-back)** to observe the occurrence of **data hazards, control hazards, and structural hazards**. A **data hazard** occurred when an instruction required the result of a previous instruction that had not yet completed, causing execution delays. A **control hazard** occurred due to branch instructions, where the processor could not determine the next instruction until the branch decision was made. A **structural hazard** occurred when multiple instructions competed for the same hardware resource, such as memory or the ALU. Initially, these hazards caused pipeline stalls, reducing throughput and increasing execution time. To overcome these issues, **stalling** was used to delay dependent instructions, **data forwarding** transferred results directly between pipeline stages without waiting for write-back, and **branch prediction** estimated the outcome of branch instructions to minimize delays. These techniques significantly reduced pipeline stalls, improved instruction flow, and increased processor efficiency.

A processor supporting **superscalar execution** was also analyzed. Unlike a traditional pipeline that executes one instruction per clock cycle, a superscalar processor can issue and execute multiple instructions simultaneously using multiple execution units. It also supports **out-of-order execution**, allowing independent instructions to execute before earlier instructions that are waiting for data, thereby improving resource utilization. In addition, **speculative execution** allows the processor to execute predicted instructions before the branch outcome is confirmed. If the prediction is correct, execution continues without interruption; otherwise, the incorrect results are discarded, and the correct instructions are executed. These advanced techniques improve instruction-level parallelism, reduce idle processor cycles, increase

throughput, and significantly enhance the overall performance of modern high-speed processors.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

ANSWER :

Superscalar, Out-of-Order, Speculative Execution, Hyper-Threading, and Simultaneous Multithreading (SMT)

A processor supporting **superscalar execution** was analyzed to understand how modern CPUs execute multiple instructions in parallel. Unlike a scalar processor, which issues one instruction per clock cycle, a superscalar processor contains multiple execution units that allow two or more independent instructions to be fetched, decoded, executed, and completed simultaneously. The processor also supports **out-of-order execution**, where instructions whose operands are ready are executed before earlier instructions that are waiting for data. This minimizes idle execution units and improves processor utilization. In addition, **speculative execution** uses branch prediction to execute instructions before the actual branch outcome is known. If the prediction is correct, execution continues without interruption; otherwise, the incorrect results are discarded and the correct instructions are re-executed.

The study was further extended to include **Hyper-Threading** and **Simultaneous Multithreading (SMT)**. Hyper-Threading allows a single physical core to appear as two logical processors, enabling two instruction streams to execute concurrently and improving CPU utilization during stalls. SMT extends this concept by allowing multiple hardware threads to share the processor's execution resources simultaneously, increasing instruction-level and thread-level parallelism. These techniques reduce idle cycles by utilizing execution units that would otherwise remain unused.

Performance analysis showed that a conventional single-issue processor completes approximately **one instruction per clock cycle**, whereas a superscalar processor can execute **two or more instructions per cycle** depending on instruction dependencies and available execution units. The combination of **out-of-order execution**, **speculative execution**, **Hyper-Threading**, and **SMT** significantly increases instruction throughput, reduces execution delays, and improves resource utilization. Although these advanced techniques require additional hardware and more complex control logic, they provide substantial performance improvements and are widely used in modern high-performance processors to achieve faster and more efficient program execution.

Code of Conduct:

I _____ YUVASRI.S (192521032) _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for

this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation